

Problem-1:

Create a **Bus** child class that inherits from the Vehicle class. The default fare charge of any vehicle is seating capacity * 100. If Vehicle is Bus instance, we need to add an extra 10% on full fare as a maintenance charge. So total fare for bus instance will become the final amount = total fare + 10% of the total fare.

Note: The bus seating capacity is 50. so the final fare amount should be 5500. You need to override the fare() method of a Vehicle class in Bus class

Input:

```
print('Total fare of vehicle with capacity(50):', Vehicles(50).fare())  
print('Total fare of bus with capacity(50):', Bus(50).fare())
```

Output:

```
Total fare of vehicle with capacity(50): 5000
```

```
Total fare of bus with capacity(50): 5500.0
```

Problem-2:

Create a Bus class that inherits from the Vehicle class. Give the capacity argument of Bus.seating_capacity() a default value of 50.

Use the following code for your parent Vehicle class.

```
class Vehicle:  
    def __init__(self, name, max_speed, mileage):  
        self.name = name  
        self.max_speed = max_speed  
        self.mileage = mileage  
    def seating_capacity(self, capacity):  
        return f"The seating capacity of a {self.name} is  
{capacity} passengers"
```

Input:

```
School_bus = Bus("School Volvo", 180, 12)
print(School_bus.seating_capacity())
```

Output:

```
The seating capacity of a School Volvo is 50 passengers
```

Problem-3:

Write a program that has a class Point. Define another class Location which has two objects (Location & Destination) of class Point. Also define a function in Location that prints the reflection of Destination on the x axis.

Input:

```
loc = Point(3, 5)
des = Point(7, 9)
place = Location(loc, des)
print(place)
place.reflection()
```

Output:

```
Location: (3 , 5)
Destination: (7 , 9)
Reflection of Location is: (3 , -5)
```

Problem-4:

Write a program that has an abstract class Polygon. Derive two classes Rectangle and Triangle from Polygon and write methods to get the details of their dimensions and hence calculate the area

Input:

```
print('Area of rectangle:', Rectangle(4,5).area())
print('Area of triangle:', Triangle(4,5).area())
```

Output:

```
Area of rectangle: 20
Area of triangle: 10.0
```

Problem-5:

Write a program with class Bill. The users have the option to pay the bill either by cheque or by cash. Use the inheritance to model this situation.

Input:

```
items = ["External Hard Disk", "RAM", "Printer", "Pen Drive"]
price = [5000, 2000, 6000, 800]

option = int(input("Would you like to pay by cheque or cash (1/2): "))

if option == 1:
    name = input("Enter the name of the bank: ")
    cno = input("Enter the cheque number: ")
    cheque = ChequePayment(items, price, cno, name)
    cheque.show_check_payment_details()
else:
    deno = [10, 20, 50, 100, 500, 2000]
    value = [1, 1, 1, 2, 4, 5]
    cash = CashPayment(items, price, deno, value)
    cash.show_cash_payment_details()
```

Output:

Problem-6:

As of now we are accessing values from dictionary with exact keys. Now we want to amend accessing values functionality. if a dict has key 1 (int) then even if we try to access values by giving '1' (1 as str) as key, we should get the same result and vice versa.

Write a class FlexibleDict upon builtin dict class with above required functionality.

Hint- dict[key] => dict.__getitem__(key)

Input:

```
fd = FlexibleDict()
fd['a'] = 100
```

```
print(fd['a']) # Like regular dict
fd[5] = 500
print(fd[5]) # Like regular dict
fd[1] = 100
print(fd['1']) # actual Key is int but still trying to access through str key.
fd['1'] = 100
print(fd[1])
```

Output:

```
100
500
100
100
```